

Step 1 — A Single LCEL Chain That Answers One Question, No Tools — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way, everything hardcoded

```
# chain.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
```

```
def build_simple_chain():
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a careful research assistant."),
        ("human", "{question}"),
    ])
    model = ChatOpenAI(model="gpt-4o-mini")
    return prompt | model | StrOutputParser()
```

```
# main.py
from chain import build_simple_chain

chain = build_simple_chain()
answer = chain.invoke({"question": "What is the capital of France?"})
print(answer)
```

Expected output:

```
The capital of France is Paris.
```

This works. It's missing type hints, doesn't use Doc01's `config / logger`, and has no way to change the model without editing this file directly — all fine for proving the chain works, worth fixing before it becomes something other steps depend on.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with Doc01's config and logger wired in

```
# chain.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import Runnable

def build_simple_chain() -> Runnable:
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a careful research assistant. Answer only from what y
        ("human", "{question}"),
    ])
    model = ChatOpenAI(model="gpt-4o-mini")
    parser = StrOutputParser()
    return prompt | model | parser
```

```
# main.py
from config import load_config
from logging_setup import get_logger
from chain import build_simple_chain

config = load_config()
logger = get_logger(__name__)

chain = build_simple_chain()
question = "What is the capital of France?"

logger.info("Asking: %s", question)
answer = chain.invoke({"question": question})
logger.info("Got answer: %s", answer)
print(answer)
```

Expected output:

The capital of France is Paris.

Difference from Basic: full type hints on `build_simple_chain()`, so anything importing it (Step 2's agent loop will) knows exactly what it gets back — a `Runnable`, which supports `.invoke()`, `.stream()`, and `.batch()` uniformly. It reuses Doc01's `config` and `logger`, which every later step in this project keeps building on. The model name is still hardcoded, though — that's what Advanced fixes.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — configurable model/temperature, plus automatic retry

```
# chain.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import Runnable
from config import Config

def build_simple_chain(config: Config) -> Runnable:
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a careful research assistant. Answer only from what y",
        ("human", "{question}"),
    ])
    model = ChatOpenAI(model=config.chat_model, temperature=config.temperature)
    chain = prompt | model | StrOutputParser()
    return chain.with_retry(stop_after_attempt=3)
```

```
# main.py
from config import load_config
from logging_setup import get_logger
from chain import build_simple_chain

config = load_config()
logger = get_logger(__name__)

chain = build_simple_chain(config)
question = "What is the capital of France?"

logger.info("Asking: %s", question)
answer = chain.invoke({"question": question})
logger.info("Got answer: %s", answer)
print(answer)
```

Expected output:

```
The capital of France is Paris.
```

`config.chat_model` and `config.temperature` need to exist on Doc01's `Config` — add them with defaults (`"gpt-4o-mini"` , `0.0`) if they aren't there yet, read from optional `.env` entries the same way the existing fields are. `.with_retry(stop_after_attempt=3)` means a transient network error or rate limit gets retried automatically up to 3 times before the exception actually reaches your code — this now matters, because Step 2's loop will call this chain repeatedly in a row, multiplying the chance of hitting a transient failure at least once.

Approach 2 — a tiny in-memory response cache for the dev loop

```
# chain.py (same build_simple_chain as Approach 1, plus this wrapper)
_response_cache: dict[str, str] = {}

def ask_with_cache(chain: Runnable, question: str) -> str:
    if question in _response_cache:
        return _response_cache[question]
    answer = chain.invoke({"question": question})
    _response_cache[question] = answer
    return answer
```

Expected output, calling `ask_with_cache(chain, "What is the capital of France?")` twice in a row in the same process:

```
The capital of France is Paris.
The capital of France is Paris.
```

The second call never hits the API — it returns instantly from `_response_cache`. This is deliberately separate from `build_simple_chain()` itself, not baked into it: caching identical questions is a dev/test convenience (rerunning your script 10 times while debugging shouldn't cost 10 API calls), not something you'd want silently caching a *user-facing* agent's answers, where the same-looking question might reasonably deserve a fresh answer later.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's chain always uses the exact model and temperature hardcoded inside it, and has no defense against a transient API failure. Approach 1 fixes both — the chain now reads its own settings, and survives the kind of one-off network hiccup a single manual test run would rarely happen to hit. Approach 2 is a separate, optional concern layered on top: it doesn't change what the chain does, it just avoids re-paying for identical calls while you're iterating on the rest of the project.

Which one should you actually write? Approach 1's configurability and retry — always; this chain gets reused by every later step in this project, so it's worth getting right once here. Approach 2's

cache is worth adding only if you notice yourself rerunning the exact same test question repeatedly during development — it's a convenience, not a requirement, and it should never wrap a chain that's actually serving real user questions, since two identical-looking questions from two different users shouldn't silently share an answer.